UNIVERSIDAD PERUANA LOS ANDES

Facultad de Ingeniería

Curso: Construcción de Software

ReservaUPLA: Sistema de Gestión de Reservas de Ambientes Universitarios

Informe técnico del caso práctico (semanas 01 a 09)

Estudiante: _____

Docente: _____

Fecha: 10 de noviembre de 2026

Huancayo, Perú

1. Problema, actores y alcance

Las reservas de laboratorios, auditorios y salas de la UPLA se hacían por mensajes, llamadas y papel: había duplicidad, solicitudes perdidas y ninguna trazabilidad. ReservaUPLA centraliza el flujo: el solicitante pide un ambiente, el sistema comprueba la disponibilidad, el administrador aprueba o rechaza y cada cambio queda registrado con quién y cuándo.

- **Actores:** Administrador (decide y gestiona), Solicitante (pide y consulta sus reservas) y Responsable de ambiente (solo consulta).
- **Alcance:** registro, validación, aprobación, consulta, disponibilidad diaria y dashboard. Fuera de alcance: correos, reservas recurrentes, calendario mensual y exportación.
- **Reglas clave:** un ambiente aprobado no admite otra reserva aprobada que se cruce (RN-01); no se reserva en el pasado ni en mantenimiento.

2. Ciclo de vida y estándares

Se eligió un modelo **incremental e iterativo**: los requisitos son estables, pero el valor se entrega por incrementos (reservar, decidir, consultar), cada uno con su prueba de cierre. Se descartaron cascada (sin feedback temprano) y espiral (exceso de gestión de riesgo para un equipo de uno).

- **Nombres y carpetas:** dominio en español, términos técnicos en inglés; backend en capas Domain, Application, Infrastructure y API, y frontend por pages, components, hooks, services y utils.
- **Dependencias:** Application nunca referencia Infrastructure; lo comprueba una prueba de arquitectura.
- **Commits y ramas:** «tipo: descripción» en español, una rama por tema; el estándar se verifica con pruebas (por ejemplo, ningún color fuera de `tokens.css`).

3. Estructura: backend, frontend y base de datos

Backend en ASP.NET Core (.NET 8) con Clean Architecture; frontend en React con Vite; base de datos PostgreSQL creada **por terminal con scripts SQL** (carpeta `database/`), sin migraciones. La API se conecta con un rol de mínimo privilegio y los secretos viven en `user-secrets`, nunca en archivos versionados.

4. Formularios y validaciones

Los formularios (nueva reserva, registrar ambiente, registrar usuario, inicio de sesión) llevan etiqueta visible, ayuda con ejemplo, error junto al campo con icono y texto, foco en el primer error y manejo completo con teclado. Las reglas se validan en **tres capas**: el frontend guía al usuario, el backend decide y la base de datos es la última barrera; los mensajes son los mismos en las tres.

Figura 1. Enviar «Nueva reserva» vacía: un mensaje junto a cada campo

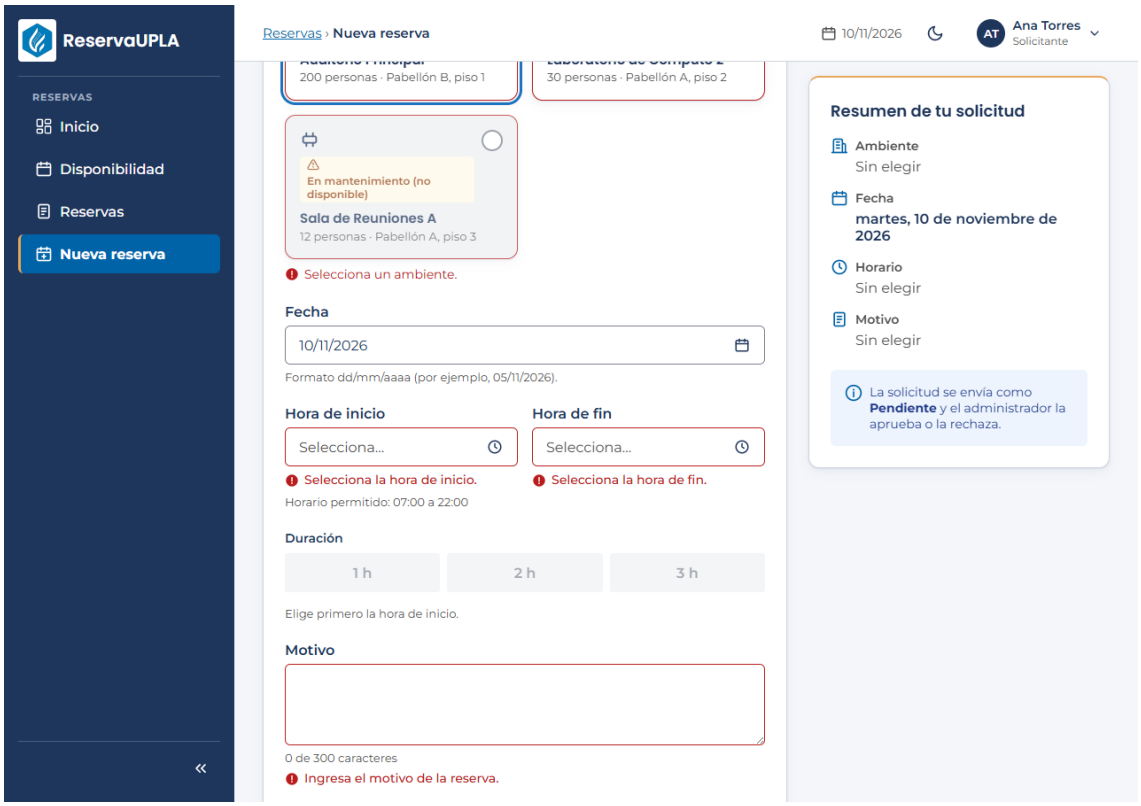
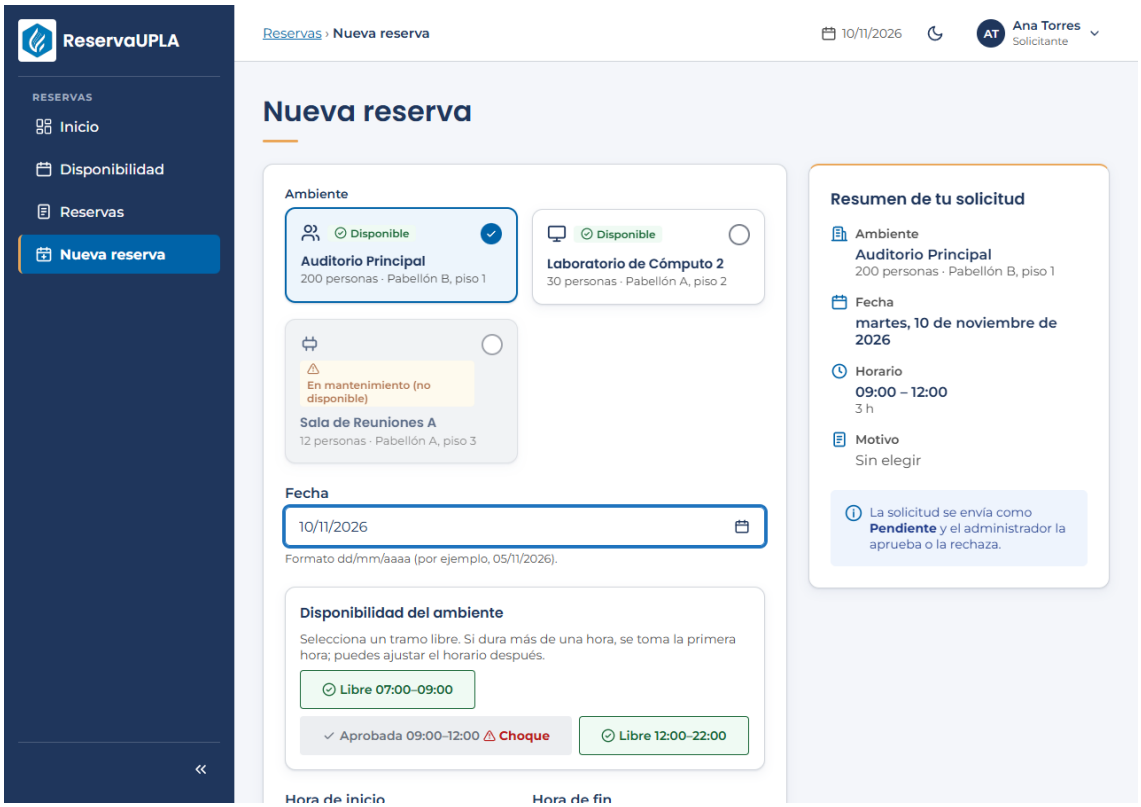


Figura 2. RN-01: aviso de disponibilidad con un tramo ocupado



La regla más delicada (no cruzar reservas aprobadas) se cubre también con una restricción de exclusión de PostgreSQL: aunque dos personas aprueben a la vez, la base impide el cruce.

5. Reutilización y refinamiento

Se construyeron los nueve componentes mínimos (TextInput, SelectInput, DateInput, TimeInput, Button, AlertMessage, ModalConfirmacion, TableList y CardIndicador) y hooks compartidos. La duplicación medida con jscpd 4.0.5, perfil de 4 líneas y 30 tokens bajó de 1,38 % a 0,93 %. Una prueba del cambio: el modo oscuro se agregó al final sin tocar ningún componente, porque todos los colores salen de un solo archivo de tokens.

El refinamiento se hizo con una red de pruebas y mirando el sistema con datos reales, lo que reveló defectos que ninguna prueba cubría (por ejemplo, «próxima reserva» mostraba una reserva cancelada). Cada defecto quedó con su prueba.

6. Salida de datos y dashboard

El Administrador ve decisiones pendientes, agenda de hoy e indicadores (total, pendientes, aprobadas, rechazadas, ambiente más usado, tasa de aprobación, ocupación de hoy y actividad reciente). El Solicitante ve su próxima reserva y sus estados, y el Responsable solo consulta.

Figura 3. Inicio del Administrador: decisiones pendientes, agenda de hoy e indicadores

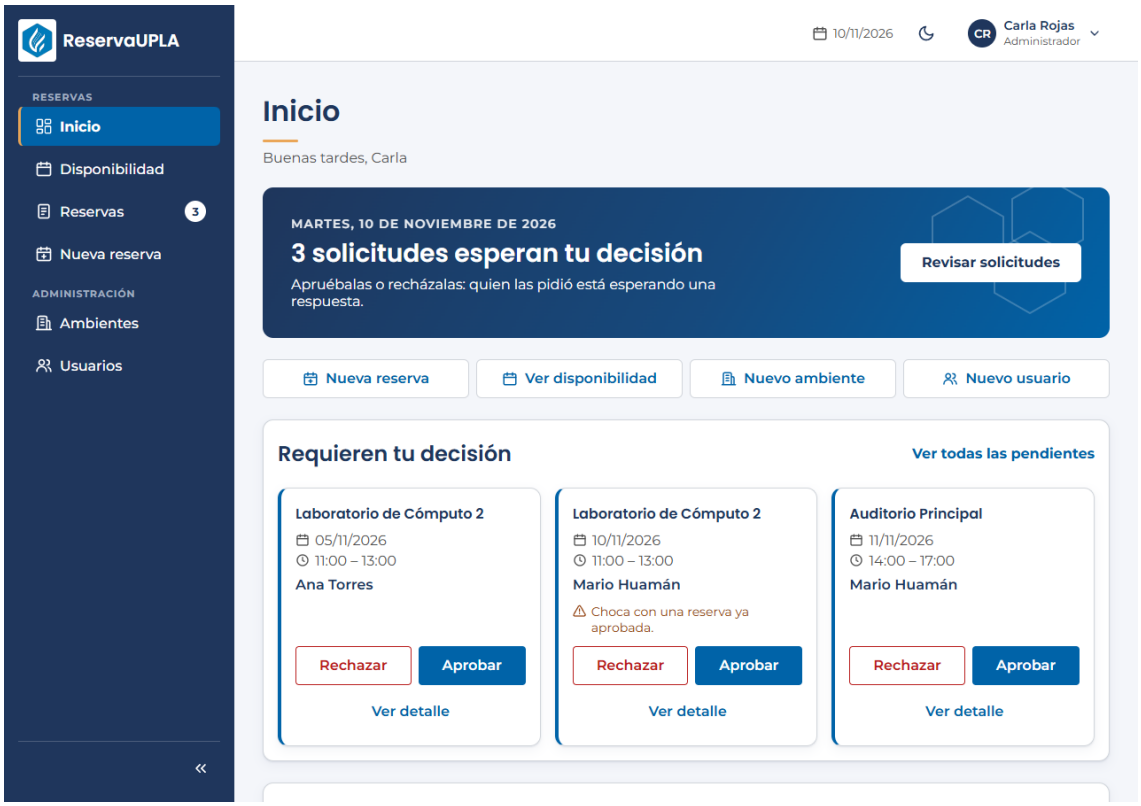


Figura 4. Listado de reservas con pestañas por estado y filtros

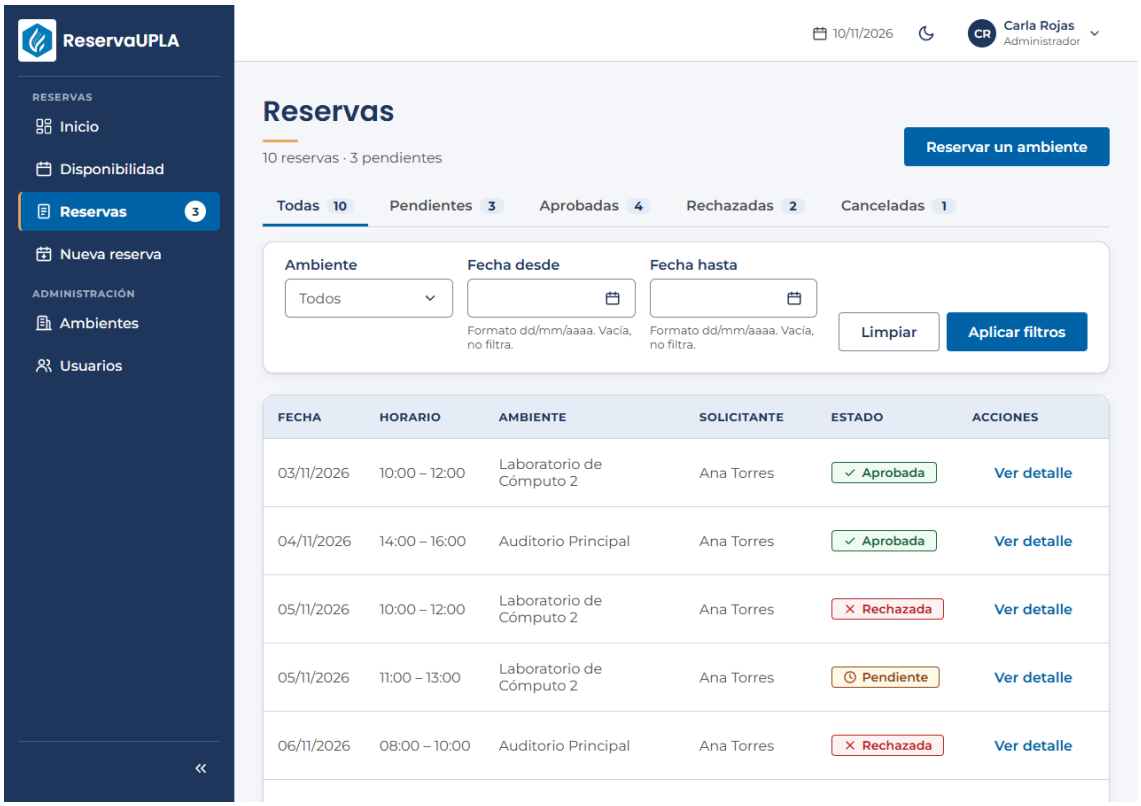


Figura 5. Vista diaria de disponibilidad por ambiente



7. Calidad y límites

Medida	Resultado	Fecha y comando
Pruebas del frontend	1375 en 62 archivos	2026-11-10, npm --prefix frontend test
Pruebas de Application	162	2026-11-10, WSL (Ubuntu), sin base de datos
Pruebas de Infrastructure	14	2026-11-10, WSL con PostgreSQL 18 real
Pruebas de la API	60 de 60	2026-11-07, Windows, con PostgreSQL real (60 de 60)
Peso del build (gzip)	JS 130,02 kB, CSS 15,79 kB	2026-11-10

Límites declarados: la última ejecución completa de las pruebas de la API, del extremo a extremo y de la accesibilidad automatizada es del 7 de noviembre; no se midió el rendimiento con carga.

8. Reflexión final

La calidad no es solo que «funcione», sino poder **cambiar sin miedo**, y las cuatro prácticas del caso aportaron algo distinto. Los **estándares** dieron orden y, convertidos en pruebas automáticas, se respetan solos. La **validación en tres capas** da confianza: el frontend ayuda, el backend decide y la base de datos garantiza. La **reutilización** hace que cada cambio se haga una sola vez. El **refinamiento** con pruebas y con datos reales encuentra lo que las pruebas no ven. Juntas mejoran la calidad (las reglas se cumplen aunque se salte el formulario), la mantenibilidad (capas con dependencias en una dirección, textos y colores en una sola fuente) y la evolución (la disponibilidad, los indicadores y el modo oscuro se añadieron sin reescribir lo anterior).

Pendiente: **[COMPLETAR POR EL ESTUDIANTE: resultados de la prueba de usabilidad con compañeros]**.